

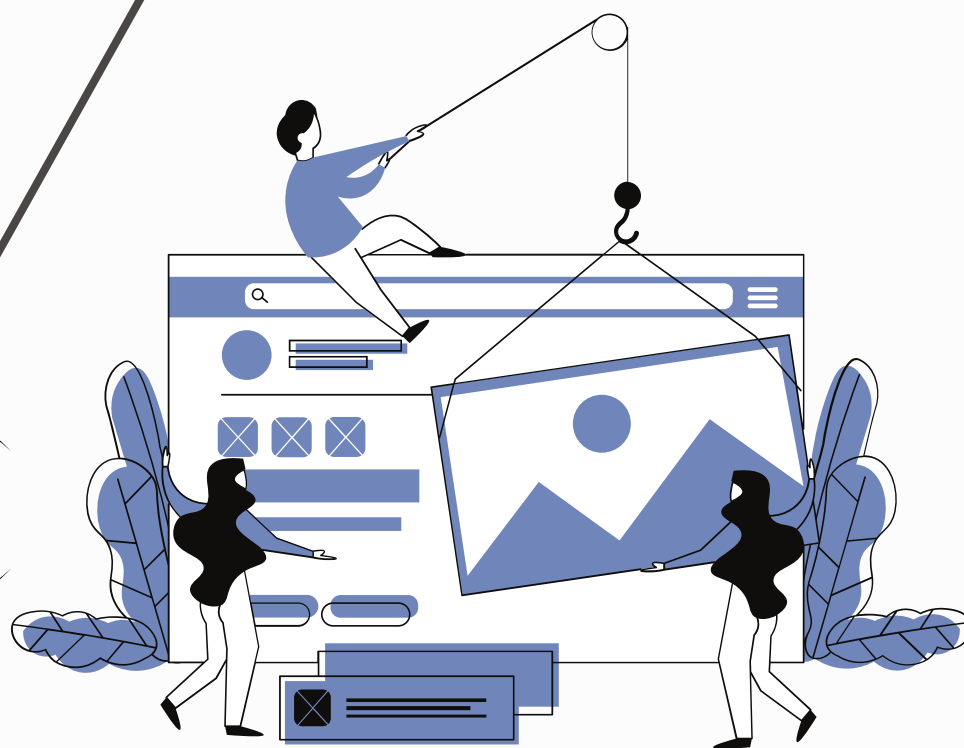
Angular



NgOnInit()



VS



Constructor()

Niyati Kaneriya



What is `constructor()`?

- A TypeScript feature, not Angular-specific
- Runs when the component class is instantiated
- Used mainly for dependency injection
- Inputs (`@Input()`) are not available yet
- Should be lightweight and free of logic
- Don't perform API calls or side effects here
- Think of it as the component's setup phase

What is `ngOnInit()` ?

- A lifecycle hook provided by Angular
- Runs after inputs are set and component is initialized
- Safe place for API calls and logic
- Use it to handle initialization that depends on inputs
- Inputs like `@Input()` are now accessible
- Ideal for fetching data or starting subscriptions
- Part of the `OnInit` interface (implements `OnInit`)

Key Differences

Feature	<code>constructor()</code>	<code>ngOnInit()</code>
Lifecycle	Class instantiation	After component init
Input access	Not available	Available
API calls?	Avoid	Best place
Purpose	Setup/DI	Initialization logic
Angular-specific?	No	Yes

Mistakes to Avoid & Best Practices

- Don't fetch data in **constructor()**
- Avoid using **@Input()** before **ngOnInit()**
- Mixing **setup (DI)** and **logic** breaks structure
- Use **constructor()** only for injecting services
- Write business logic inside **ngOnInit()**
- Keep lifecycle order: **Setup** » **Init**
- Use implements OnInit for *clarity & maintainability*

Real Example

```
example-component.ts

// constructor: for dependency injection only
constructor(private userService: UserService) {
  console.log('Constructor: Service injected');
}

// ngOnInit: ideal place for logic
ngOnInit() {
  this.userService.getUser().subscribe(data => {
    this.user = data;
    console.log('ngOnInit: User data loaded');
  });
}
```

- Inputs aren't ready in constructor
- ngOnInit() is lifecycle-aware
- Follow this pattern for clean code

Summary

- Use **constructor** for **initial setup** only
- Use **ngOnInit()** for **logic and input-aware code**
- Don't mix responsibilities
- Clean lifecycle usage = scalable apps